

Lab 2: Geometry Processing

王唐欣宇 — 2400017808

October 1, 2025

1 Task 1: Loop Mesh Subdivision

I implemented the Loop subdivision algorithm as described in the lecture slides. The implementation is straightforward, and I have added comments to the code for clarity.

It is worth noting that the ordering of the newly generated vertices is consistent with the ordering of the original vertices. For example, if the original vertices are ordered counterclockwise, the new vertices will also be ordered counterclockwise, as illustrated in Figure 1a.

The results of the Loop subdivision algorithm are shown in Figures 1b, 1c, and 1d.

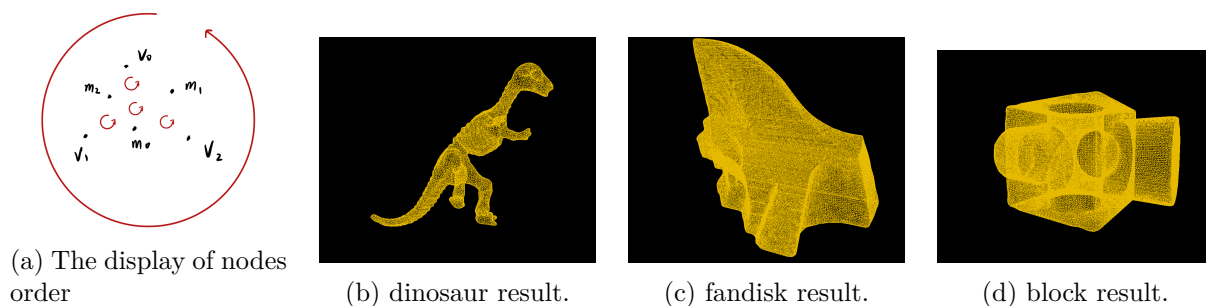


Figure 1: The Result of Task 1.

2 Task 2: Spring-Mass Mesh Parameterization

I implemented the spring-mass mesh parameterization algorithm as described in the lecture slides. Since the target parameterization domain is the unit square $[0, 1] \times [0, 1]$, I mapped the boundary vertices of the mesh onto the perimeter of this square. It should be noted that the order of the boundary vertices is crucial. To establish this order, I first collected all boundary vertices. Then, starting from an arbitrary boundary vertex, I traversed the boundary edges to generate an ordered sequence of vertices. Finally, I mapped these ordered vertices to the boundary of the unit square.

After this setup, the process is an iterative relaxation where the boundary vertices remain fixed. The result is shown in Figure 2.

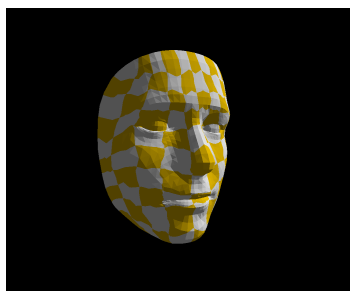
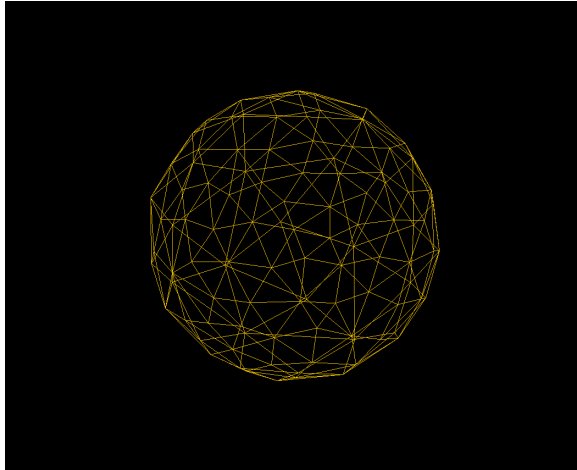


Figure 2: The Result of Task 2.

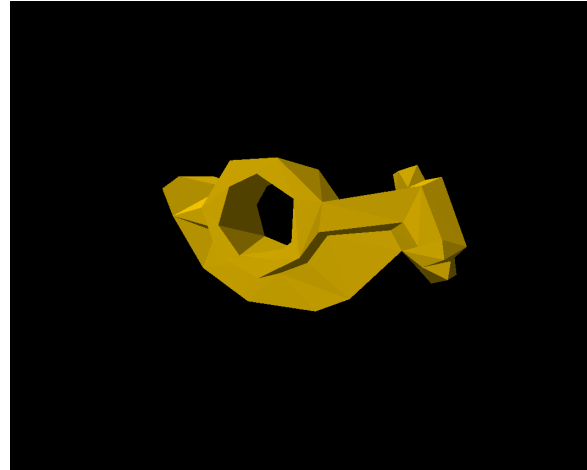
3 Task 3: Mesh Simplification

I implemented the mesh simplification algorithm based on quadric error metrics, as described in the lecture slides. A key aspect of the implementation is efficiently updating the costs of affected edge collapses. After an edge is collapsed, the Q-matrices of the vertices in the 1-ring neighborhood of the collapsed vertex are affected. I identify all edges connected to these neighboring vertices and update their collapse costs in the vector.

The results are shown in Figure 3.



(a) sphere result.

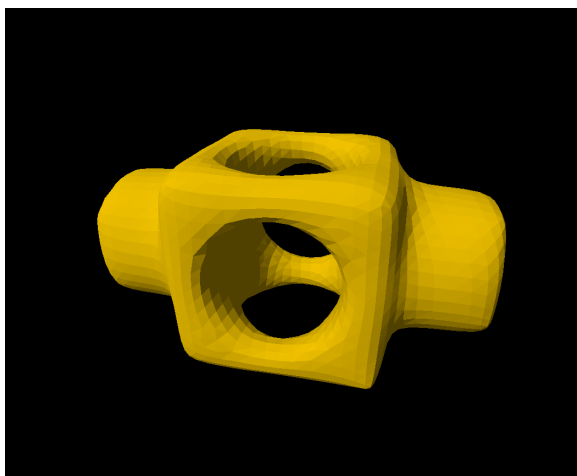


(b) rocker Result.

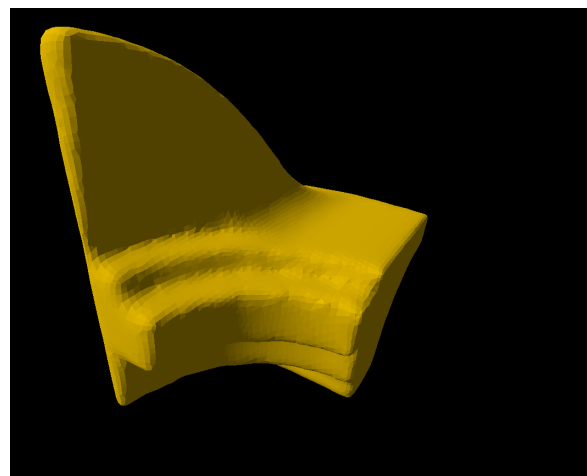
Figure 3: The Result of Task 3.

4 Task 4: Mesh Smoothing

I implemented the mesh smoothing algorithm using the cotangent Laplacian, as described in the lecture slides. It is important to handle cases where cotangent values can become very large or small, which can lead to numerical instability. I added checks to clamp these values within a reasonable range. The results are shown in Figure 4.



(a) block result.



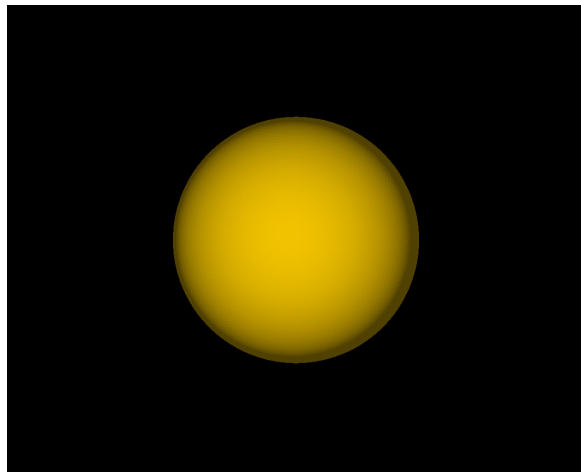
(b) fandisk Result.

Figure 4: The Result of Task 4.

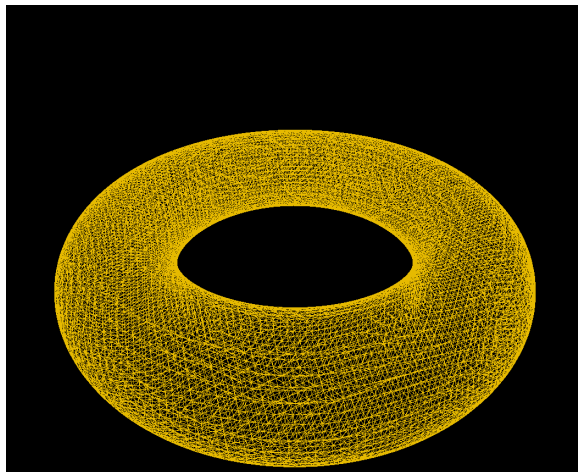
5 Task 5: Marching Cubes

I implemented the marching cubes algorithm as described in the lecture slides. A tricky part of the implementation is ensuring that vertices generated along the same edge are reused, not duplicated. To solve this, I assigned a unique ID to each edge in the grid and used a map to store the generated vertex for that edge. This ensures that each vertex is created only once.

The results are shown in Figure 5.



(a) sphere result.



(b) torus Result.

Figure 5: The Result of Task 5.